

dmn_insula_trait_anxiety_pipeline

May 20, 2026

1 Example fMRI Pipeline: Does trait anxiety predict stronger DMN–insula connectivity?

Research question:

> Does trait anxiety predict stronger functional connectivity between the Default Mode Network (DMN) and the insula?

This notebook is a **teaching pipeline** that transforms a neuroscience idea into a reproducible analysis plan.

It is written for VS Code/Jupyter on Ubuntu, but it should also run on Windows or macOS.

1.1 Big picture

We will move through the full logic:

1. Find a dataset with STAI trait anxiety scores.
 2. Identify subjects with usable fMRI + anxiety metadata.
 3. Load BIDS-style data.
 4. Extract ROIs for DMN and insula.
 5. Compute functional connectivity.
 6. Compare high- vs low-trait-anxiety groups.
 7. Visualize the result.
 8. Interpret the biological meaning.
-

1.2 Important teaching note

This notebook has two modes:

1.2.1 Mode A — real BIDS data

Use this when you have downloaded a real BIDS dataset and have preprocessed functional images.

1.2.2 Mode B — simulated data

Use this immediately to learn the workflow even before the real dataset is downloaded.

The simulated mode is not evidence for the hypothesis. It is a sandbox that lets you practice the pipeline structure.

1.3 Dataset target

A reasonable target dataset family for this question is the **Amsterdam Open MRI Collection (AOMIC)** because published descriptions note that it includes MRI plus questionnaire measures, including STAI-type anxiety measures.

For a real project, you would verify:

- Is there a `participants.tsv` or phenotype file?
- Is there a trait anxiety column?
- Are there resting-state or naturalistic fMRI runs?
- Are the fMRI files already preprocessed?
- Are confounds/motion files available?

For this first notebook, we will write code that expects a BIDS-like folder, but we will also include simulated fallback data so the full notebook runs.

```
[1]: # =====
# 0. Setup
# =====

# If these packages are missing, install them in your VS Code terminal:
# pip install pandas numpy matplotlib scipy nibabel nilearn scikit-learn

from pathlib import Path
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

# Optional neuroimaging imports.
# The notebook can still run in simulated mode if these are not installed.
try:
    import nibabel as nib
    from nilearn import datasets, plotting
    from nilearn.maskers import NiftiLabelsMasker
    HAVE_NILEARN = True
except Exception as e:
    HAVE_NILEARN = False
    print("Nilearn/nibabel not available. Simulated mode will still work.")
    print("Import issue:", e)

RANDOM_SEED = 42
```

```
np.random.seed(RANDOM_SEED)
```

1.4 1. Define the initial question

Before touching code, state the scientific idea in a form that can become an analysis.

1.4.1 Conceptual hypothesis

People with higher trait anxiety may show stronger coupling between:

- **DMN:** internally oriented thought, self-referential processing, autobiographical thought.
- **Insula / salience network:** interoception, bodily feeling, salience detection, switching between internal and external attention.

1.4.2 Testable version

Trait anxiety score predicts mean functional connectivity between DMN ROIs and insula ROIs.

1.4.3 Statistical version

We can test either:

1. **Continuous prediction:** Correlate STAI trait score with DMN–insula connectivity.
2. **Group comparison:** Compare high-STAI vs low-STAI subjects.

Both are useful. The continuous version usually preserves more information.

```
[2]: # =====  
# 1. Project paths  
# =====  
  
# Change this path when you have a real BIDS dataset.  
# Example:  
# BIDS_ROOT = Path("/home/john/data/aomic")  
BIDS_ROOT = Path("/path/to/your/BIDS_dataset")  
  
# Preprocessed fMRI is often stored in derivatives, for example:  
# /dataset/derivatives/fmriprep/sub-001/func/...  
DERIV_ROOT = BIDS_ROOT / "derivatives" / "fmriprep"  
  
# Set this to True only when your real paths exist.  
USE_REAL_BIDS_DATA = BIDS_ROOT.exists()  
  
print("BIDS root:", BIDS_ROOT)  
print("Use real BIDS data?", USE_REAL_BIDS_DATA)
```

```
BIDS root: /path/to/your/BIDS_dataset  
Use real BIDS data? False
```

1.5 2. Find and inspect STAI metadata

In BIDS datasets, subject-level variables often live in:

- `participants.tsv`
- `phenotype/*.tsv`
- dataset-specific questionnaire files

We want a table with at least:

- `participant_id`
- trait anxiety score, often named something like `STAI_Trait`, `stai_trait`, `STAI_T_trait_anxiety`, or `STAI_Trait_Total`.

Because real datasets vary in column names, we write helper code to search likely files and likely columns.

```
[3]: # =====  
# 2. Setup: imports, folders, and helper functions  
# =====  
  
from pathlib import Path  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
  
# Reproducibility: makes simulated results the same each time  
np.random.seed(42)  
  
# -----  
# Project folders  
# -----  
  
PROJECT_ROOT = Path.home() / "Data" / "Research" / "connectome"  
  
DATA_DIR = PROJECT_ROOT / "data"  
BIDS_DIR = DATA_DIR / "bids"  
RESULTS_DIR = PROJECT_ROOT / "results"  
FIGURES_DIR = PROJECT_ROOT / "figures"  
  
RESULTS_DIR.mkdir(parents=True, exist_ok=True)  
FIGURES_DIR.mkdir(parents=True, exist_ok=True)  
  
print("Project root:", PROJECT_ROOT)  
print("BIDS data folder:", BIDS_DIR)  
print("Results folder:", RESULTS_DIR)  
print("Figures folder:", FIGURES_DIR)
```

```

# -----
# Possible STAI / trait anxiety column names
# -----

POSSIBLE_STAI_COLUMNS = [
    "STAI_Trait",
    "STAI_Trait_Total",
    "trait_anxiety",
    "TraitAnxiety",
    "stai_trait_total",
    "STAIT",
    "STAI-Y2",
]

# -----
# Helper function: find metadata tables
# -----

def find_metadata_tables(bids_root):
    """
    Search for likely participant/metadata tables in a BIDS dataset.

    Why:
    BIDS datasets often store behavioural or questionnaire data in .tsv or .csv
    ↪ files.
    We are looking for a table that might contain trait anxiety / STAI scores.
    """

    bids_root = Path(bids_root)

    if not bids_root.exists():
        return []

    tables = []

    for pattern in ["*.tsv", "*.csv"]:
        tables.extend(bids_root.rglob(pattern))

    return tables

# -----
# Helper function: find a STAI-like column
# -----

def find_stai_column(df):

```

```

"""
Try to identify a trait anxiety column in a dataframe.

Why:
Different datasets may use different column names for the same concept.
"""

lower_cols = [col.lower() for col in df.columns]

for possible_name in POSSIBLE_STAI_COLUMNS:
    if possible_name.lower() in lower_cols:
        idx = lower_cols.index(possible_name.lower())
        return df.columns[idx]

return None

# -----
# Helper function: Fisher z transform
# -----

def fisher_z(r):
    """
    Convert correlation values to Fisher z values.

    Why:
    Correlation coefficients are not normally distributed near -1 or +1.
    Fisher z makes them better behaved for statistical comparisons.
    """

    r = np.clip(r, -0.999999, 0.999999)
    return np.arctanh(r)

```

Project root: /home/john-walkey/Data/Research/connectome
 BIDS data folder: /home/john-walkey/Data/Research/connectome/data/bids
 Results folder: /home/john-walkey/Data/Research/connectome/results
 Figures folder: /home/john-walkey/Data/Research/connectome/figures

1.6 3. Create or load participant table

If real metadata is available, use it.

If not, simulate a subject table with:

- 60 subjects
- STAI trait anxiety scores
- group labels based on upper/lower tertiles

This lets you practice the analysis logic now.

```
[4]: # =====
# 3. Participant metadata
# =====

def simulate_participants(n_subjects=60):
    """
    Create a teaching dataset.

    STAI trait scores are simulated around a plausible adult range.
    These are NOT real participant data.
    """
    participant_id = [f"sub-{i:03d}" for i in range(1, n_subjects + 1)]
    stai_trait = np.clip(np.random.normal(loc=42, scale=10, size=n_subjects),
↪20, 80)

    return pd.DataFrame({
        "participant_id": participant_id,
        "stai_trait": stai_trait
    })

if USE_REAL_BIDS_DATA and "participants" in globals() and stai_col is not None:
    participants = participants[["participant_id", stai_col]].copy()
    participants = participants.rename(columns={stai_col: "stai_trait"})
    participants["stai_trait"] = pd.to_numeric(participants["stai_trait"],
↪errors="coerce")
    participants = participants.dropna(subset=["stai_trait"])
else:
    participants = simulate_participants(n_subjects=60)

low_cut = participants["stai_trait"].quantile(1/3)
high_cut = participants["stai_trait"].quantile(2/3)

participants["anxiety_group"] = "middle"
participants.loc[participants["stai_trait"] <= low_cut, "anxiety_group"] = "low"
participants.loc[participants["stai_trait"] >= high_cut, "anxiety_group"] =
↪"high"

participants.head()
```

```
[4]: participant_id  stai_trait  anxiety_group
0         sub-001    46.967142             high
1         sub-002    40.617357          middle
2         sub-003    48.476885             high
3         sub-004    57.230299             high
4         sub-005    39.658466          middle
```

```
[5]: # Inspect subject counts
participants["anxiety_group"].value_counts()
```

```
[5]: anxiety_group
high      20
middle    20
low       20
Name: count, dtype: int64
```

1.7 4. Identify subjects with usable fMRI

For real BIDS data, this step checks which subjects have functional images.

A common preprocessed fMRI filename looks like:

sub-001_task-rest_space-MNI152NLin2009cAsym_desc-preproc_bold.nii.gz

We also usually want confounds:

sub-001_task-rest_desc-confounds_timeseries.tsv

For this teaching notebook, we write the function but only use it when a real dataset exists.

```
[6]: # =====
# 4. Find preprocessed fMRI files
# =====

def find_preprocessed_bold(deriv_root: Path, participant_id: str):
    """
    Find a preprocessed BOLD file for one subject.

    Functional connectivity should usually be computed from preprocessed data,
    not raw BOLD. Preprocessing handles motion correction, normalization,
    slice timing issues, and nuisance variables depending on pipeline.
    """
    subj_dir = deriv_root / participant_id / "func"

    if not subj_dir.exists():
        return None

    patterns = [
        f"{participant_id}_task-rest_*desc-preproc_bold.nii.gz",
        f"{participant_id}_task-rest_*bold.nii.gz",
        f"{participant_id}_*desc-preproc_bold.nii.gz",
    ]

    for pattern in patterns:
        matches = sorted(subj_dir.glob(pattern))
        if matches:
            return matches[0]
```



```

    return None

if USE_REAL_BIDS_DATA:
    participants["bold_file"] = participants["participant_id"].apply(
        lambda sid: find_preprocessed_bold(DERIV_ROOT, sid)
    )
    usable = participants.dropna(subset=["bold_file"]).copy()
else:
    usable = participants.copy()
    usable["bold_file"] = "SIMULATED"

print("Usable subjects:", len(usable))
usable.head()

```

Usable subjects: 60

```

[6]: participant_id  stai_trait  anxiety_group  bold_file
0      sub-001      46.967142           high  SIMULATED
1      sub-002      40.617357        middle  SIMULATED
2      sub-003      48.476885           high  SIMULATED
3      sub-004      57.230299           high  SIMULATED
4      sub-005      39.658466        middle  SIMULATED

```

1.8 5. Define ROIs: DMN and insula

There are many correct ways to define ROIs. For a real paper, this choice must be justified carefully.

Possible options:

1. **Atlas-based ROIs** — Harvard-Oxford, Schaefer, Yeo networks.
2. **Coordinate spheres** — PCC, mPFC, angular gyrus, anterior insula.
3. **Network atlas labels** — Yeo DMN plus atlas-defined insula.

For a beginner pipeline, we will use a simple atlas approach.

The code below shows how to fetch an atlas with Nilearn. In simulated mode, we skip image extraction and simulate ROI time series directly.

```

[7]: # =====
# 5. Atlas setup
# =====

if HAVE_NILEARN:
    # Harvard-Oxford cortical atlas is easy for teaching.
    atlas = datasets.fetch_atlas_harvard_oxford("cort-maxprob-thr25-2mm")
    atlas_img = atlas.maps
    atlas_labels = atlas.labels

```

```

print("Number of atlas labels:", len(atlas_labels))
print("Example labels:")
for label in atlas_labels[:15]:
    print(" -", label)
else:
    atlas = None
    atlas_img = None
    atlas_labels = []
    print("Skipping atlas fetch because nilearn is not available.")

```

[fetch_atlas_harvard_oxford] Dataset found in /home/john-walkey/nilearn_data/fsl
Number of atlas labels: 49

Example labels:

- Background
- Frontal Pole
- Insular Cortex
- Superior Frontal Gyrus
- Middle Frontal Gyrus
- Inferior Frontal Gyrus, pars triangularis
- Inferior Frontal Gyrus, pars opercularis
- Precentral Gyrus
- Temporal Pole
- Superior Temporal Gyrus, anterior division
- Superior Temporal Gyrus, posterior division
- Middle Temporal Gyrus, anterior division
- Middle Temporal Gyrus, posterior division
- Middle Temporal Gyrus, temporooccipital part
- Inferior Temporal Gyrus, anterior division

```

[8]: # Find candidate labels for DMN and insula.
# This is intentionally transparent so you can inspect whether the labels make_
↳sense.

if HAVE_NILEARN:
    label_df = pd.DataFrame({
        "index": range(len(atlas_labels)),
        "label": atlas_labels
    })

    dmnn_keywords = [
        "Cingulate", "Precuneous", "Frontal Medial",
        "Paracingulate", "Angular", "Temporal Pole"
    ]
    insula_keywords = ["Insular", "Insula"]

    candidate_dmnn = label_df[

```

```

        label_df["label"].str.contains("|".join(dmn_keywords), case=False,
↪regex=True)
    ]

    candidate_insula = label_df[
        label_df["label"].str.contains("|".join(insula_keywords), case=False,
↪regex=True)
    ]

    print("Candidate DMN-related atlas labels:")
    display(candidate_dmn)

    print("Candidate insula labels:")
    display(candidate_insula)
else:
    print("In simulated mode, we will use named ROIs without real atlas
↪extraction.")

```

Candidate DMN-related atlas labels:

	index	label
5	5	Inferior Frontal Gyrus, pars triangularis
8	8	Temporal Pole
21	21	Angular Gyrus
25	25	Frontal Medial Cortex
28	28	Paracingulate Gyrus
29	29	Cingulate Gyrus, anterior division
30	30	Cingulate Gyrus, posterior division
31	31	Precuneous Cortex

Candidate insula labels:

	index	label
2	2	Insular Cortex

1.9 6. Extract ROI time series

For each subject, we want a matrix like:

timepoints × ROIs

Example columns:

PCC, mPFC, Angular_L, Angular_R, Insula_L, Insula_R

Functional connectivity is usually estimated by asking:

Do two brain regions fluctuate together over time?

If yes, their correlation is higher.

```

[9]: # =====
# 6A. Real-data ROI extraction function
# =====

def extract_roi_timeseries_real(bold_file, atlas_img, selected_label_indices):
    """
    Extract ROI time series from a real preprocessed BOLD file.

    Parameters
    -----
    bold_file : path
        Preprocessed functional image.
    atlas_img : Nifti image
        Atlas image where voxel values identify labels.
    selected_label_indices : list[int]
        Atlas label numbers to extract.

    Returns
    -----
    time_series : ndarray
        Shape: timepoints x selected_ROIs
    """
    masker = NiftiLabelsMasker(
        labels_img=atlas_img,
        labels=selected_label_indices,
        standardize="zscore_sample",
        detrend=True,
        verbose=0
    )

    time_series = masker.fit_transform(str(bold_file))
    return time_series

# =====
# 6B. Simulated ROI time series function
# =====

ROI_NAMES = ["PCC_DMN", "mPFC_DMN", "Angular_L_DMN", "Angular_R_DMN",
    ↪ "Insula_L", "Insula_R"]
DMN_ROIS = ["PCC_DMN", "mPFC_DMN", "Angular_L_DMN", "Angular_R_DMN"]
INSULA_ROIS = ["Insula_L", "Insula_R"]

def simulate_roi_timeseries(stai_trait, n_timepoints=180):
    """
    Simulate ROI time series where DMN-insula coupling increases with trait_
    ↪ anxiety.

```

This creates a controlled sandbox where the pipeline should recover a relationship between anxiety and connectivity.

This is NOT biological evidence.

```
"""
normalized = (stai_trait - 20) / (80 - 20)
coupling = 0.05 + 0.45 * normalized

dmn_signal = np.random.normal(size=n_timepoints)
insula_signal = coupling * dmn_signal + np.sqrt(1 - coupling**2) * np.
↪random.normal(size=n_timepoints)

data = {}
for roi in DMN_ROIS:
    data[roi] = dmn_signal + np.random.normal(scale=0.8, size=n_timepoints)

for roi in INSULA_ROIS:
    data[roi] = insula_signal + np.random.normal(scale=0.8,
↪size=n_timepoints)

return pd.DataFrame(data)
```

1.10 7. Compute functional connectivity matrix

A functional connectivity matrix is simply a correlation matrix among ROI time series.

If we have 6 ROIs, the FC matrix is 6×6 .

Then we summarize our main target:

Mean correlation between all DMN ROIs and all insula ROIs.

This gives one DMN-insula connectivity score per subject.

```
[10]: # =====
# 7. FC calculation helpers
# =====

def compute_fc_matrix(timeseries_df: pd.DataFrame):
    """Compute Pearson correlation among ROI time series."""
    return timeseries_df.corr()

def mean_between_network_fc(fc_matrix: pd.DataFrame, network_a, network_b):
    """Average connectivity between two sets of ROIs."""
    values = []

    for roi_a in network_a:
```

```

        for roi_b in network_b:
            values.append(fc_matrix.loc[roi_a, roi_b])

    return np.mean(values)

def fisher_z(r):
    """
    Convert correlation r to Fisher z.

    Correlations are bounded between -1 and 1. Fisher z makes them
    more suitable for many standard statistical tests.
    """
    r = np.clip(r, -0.999999, 0.999999)
    return np.arctanh(r)

```

```

[11]: # =====
# 7. Run FC calculation for each subject
# =====

results = []
example_fc = None

for _, row in usable.iterrows():
    sid = row["participant_id"]
    stai = row["stai_trait"]

    if USE_REAL_BIDS_DATA and HAVE_NILEARN:
        # Real-data template:
        # 1. Inspect candidate_dmn and candidate_insula above.
        # 2. Choose label indices deliberately.
        # 3. Extract time series.
        # 4. Rename columns to match DMN_ROIS and INSULA_ROIS.
        raise NotImplementedError(
            "Real extraction template reached. Inspect atlas labels and set_
selected_label_indices first."
        )
    else:
        ts = simulate_roi_timeseries(stai_trait=stai)
        fc = compute_fc_matrix(ts)

    dmn_insula_r = mean_between_network_fc(fc, DMN_ROIS, INSULA_ROIS)
    dmn_insula_z = fisher_z(dmn_insula_r)

    results.append({
        "participant_id": sid,
        "stai_trait": stai,

```

```

        "anxiety_group": row["anxiety_group"],
        "dmn_insula_r": dmn_insula_r,
        "dmn_insula_z": dmn_insula_z
    })

    if example_fc is None:
        example_fc = fc.copy()

fc_results = pd.DataFrame(results)
fc_results.head()

```

```

[11]: participant_id  stai_trait  anxiety_group  dmn_insula_r  dmn_insula_z
0      sub-001      46.967142           high      0.070911      0.071030
1      sub-002      40.617357        middle      0.107560      0.107978
2      sub-003      48.476885           high      0.205475      0.208443
3      sub-004      57.230299           high      0.191802      0.194207
4      sub-005      39.658466        middle      0.126784      0.127470

```

1.11 8. Visualize one subject's FC matrix

This checks whether the output looks sensible.

In a real analysis, this step helps catch problems such as:

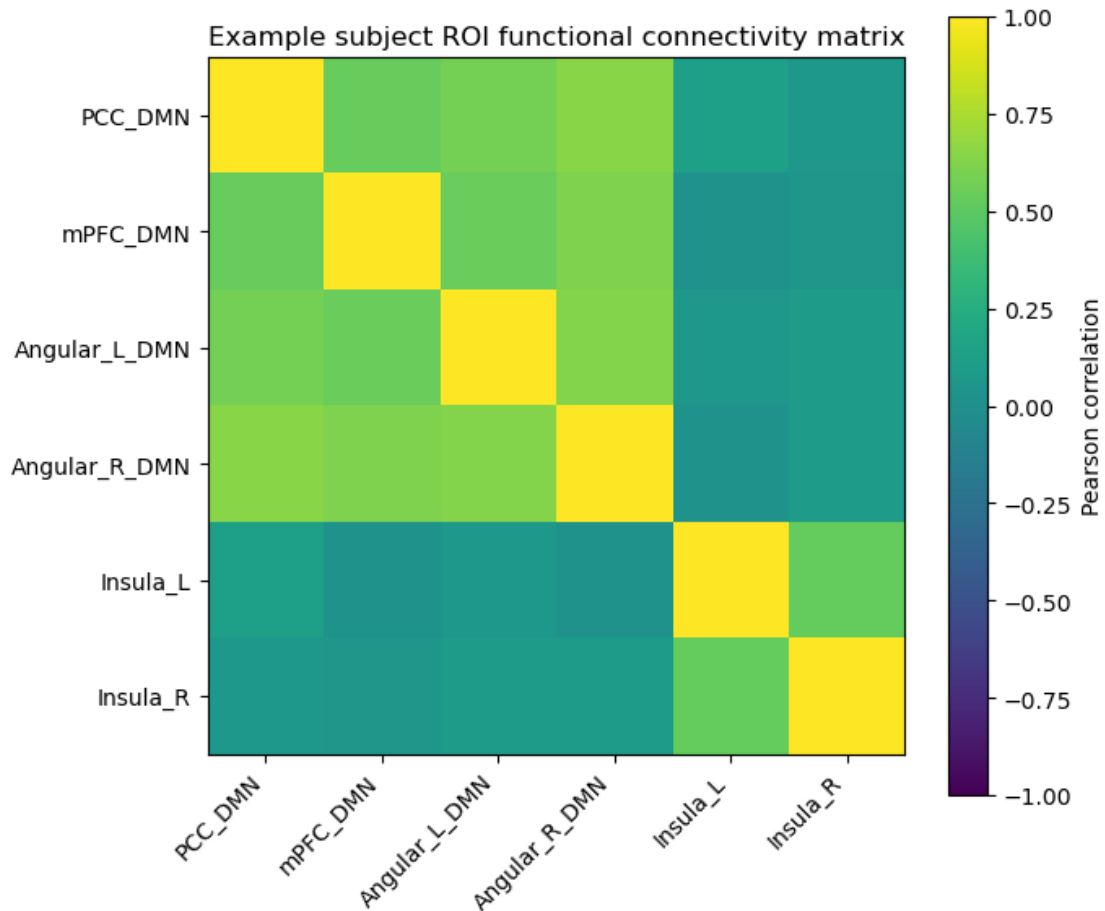
- wrong ROI labels
- all-zero time series
- strange correlations
- bad subject files

```

[12]: # =====
# 8. Plot example FC matrix
# =====

plt.figure(figsize=(7, 6))
plt.imshow(example_fc, vmin=-1, vmax=1)
plt.colorbar(label="Pearson correlation")
plt.xticks(range(len(example_fc.columns)), example_fc.columns, rotation=45,
            ha="right")
plt.yticks(range(len(example_fc.index)), example_fc.index)
plt.title("Example subject ROI functional connectivity matrix")
plt.tight_layout()
plt.show()

```



1.12 9. Test the main hypothesis: continuous STAI prediction

Now we test:

Do subjects with higher trait anxiety have stronger DMN-insula connectivity?

We use correlation first because STAI is a continuous score.

- x-axis: STAI trait anxiety
- y-axis: DMN-insula FC

```
[13]: # =====
# 9. Correlation test
# =====

r, p = stats.pearsonr(fc_results["stai_trait"], fc_results["dmn_insula_z"])

print("Correlation between STAI trait and DMN-insula FC:")
print(f"r = {r:.3f}")
print(f"p = {p:.5f}")
```


Correlation between STAI trait and DMN-insula FC:

$r = 0.587$

$p = 0.00000$

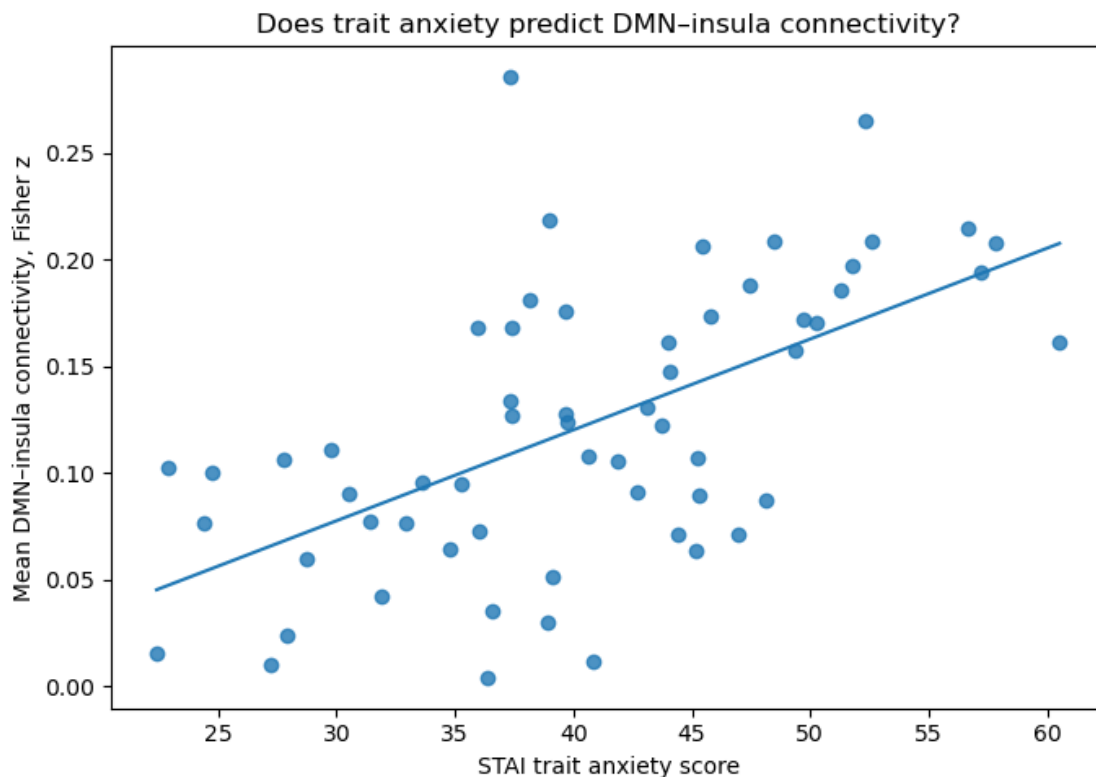
```
[14]: # Scatter plot with regression line

x = fc_results["stai_trait"].values
y = fc_results["dmn_insula_z"].values

slope, intercept, r_value, p_value, std_err = stats.linregress(x, y)
x_line = np.linspace(x.min(), x.max(), 100)
y_line = intercept + slope * x_line

plt.figure(figsize=(7, 5))
plt.scatter(x, y, alpha=0.8)
plt.plot(x_line, y_line)
plt.xlabel("STAI trait anxiety score")
plt.ylabel("Mean DMN-insula connectivity, Fisher z")
plt.title("Does trait anxiety predict DMN-insula connectivity?")
plt.tight_layout()
plt.show()

print(f"Slope = {slope:.4f}")
print(f"Intercept = {intercept:.4f}")
print(f"r = {r_value:.3f}, p = {p_value:.5f}")
```



```
Slope = 0.0043
Intercept = -0.0500
r = 0.587, p = 0.00000
```

1.13 10. Compare high- vs low-trait-anxiety groups

Now we do the simpler group comparison:

- Low group = bottom third of STAI scores
- High group = top third of STAI scores
- Middle group is excluded for a clean contrast

This is useful for visualization, but it throws away information compared with the continuous analysis.

```
[15]: # =====
# 10. Group comparison
# =====

group_df = fc_results[fc_results["anxiety_group"].isin(["low", "high"])].copy()

low_values = group_df.loc[group_df["anxiety_group"] == "low", "dmn_insula_z"]
high_values = group_df.loc[group_df["anxiety_group"] == "high", "dmn_insula_z"]

t_stat, p_ttest = stats.ttest_ind(high_values, low_values, equal_var=False)

print("High vs low trait anxiety comparison:")
print(f"Low mean = {low_values.mean():.3f}")
print(f"High mean = {high_values.mean():.3f}")
print(f"t = {t_stat:.3f}")
print(f"p = {p_ttest:.5f}")
```

```
High vs low trait anxiety comparison:
Low mean = 0.071
High mean = 0.167
t = 6.242
p = 0.00000
```

```
[16]: # Bar plot with individual points

summary = group_df.groupby("anxiety_group")["dmn_insula_z"].agg(["mean",
↪ "sem"]).loc[["low", "high"]]

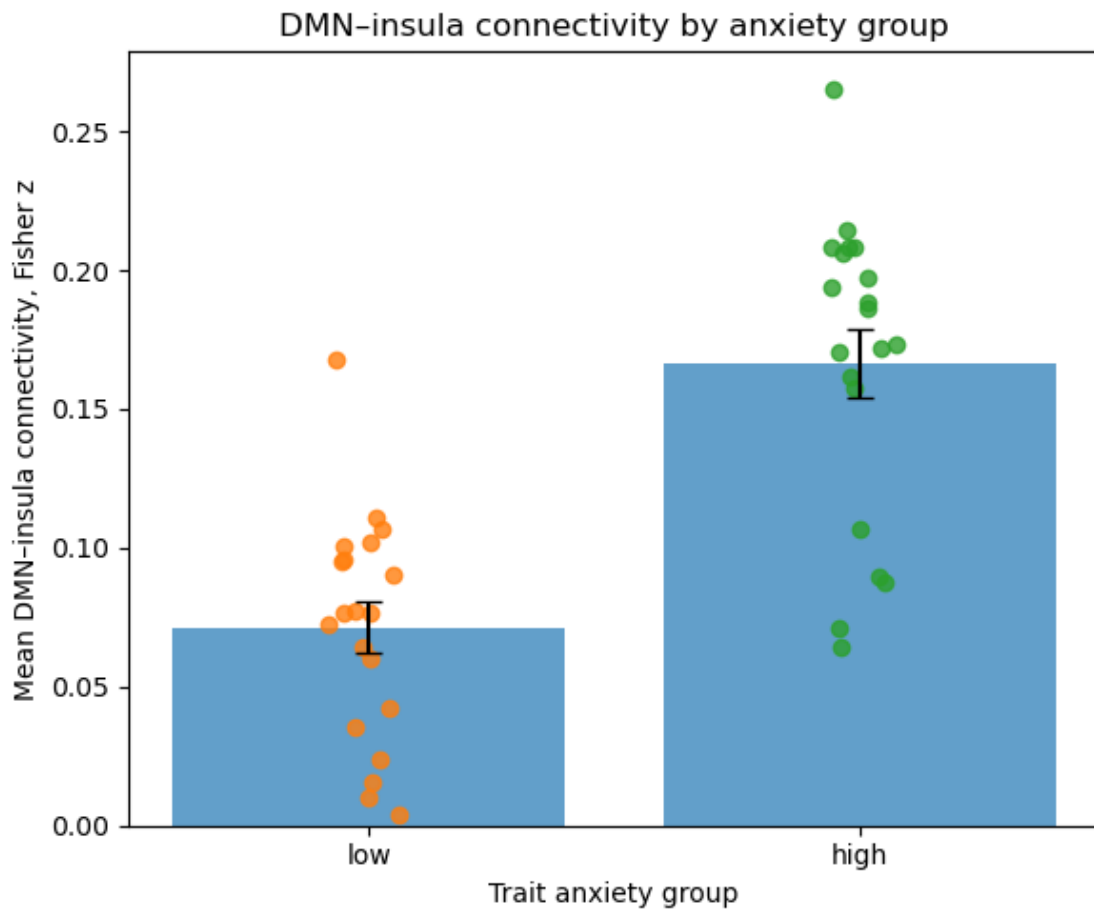
plt.figure(figsize=(6, 5))
plt.bar(summary.index, summary["mean"], yerr=summary["sem"], capsize=5, alpha=0.
↪ 7)
```

```

for i, group in enumerate(summary.index):
    vals = group_df.loc[group_df["anxiety_group"] == group, "dmn_insula_z"].
    ↪ values
    jitter = np.random.normal(loc=i, scale=0.04, size=len(vals))
    plt.scatter(jitter, vals, alpha=0.8)

plt.xlabel("Trait anxiety group")
plt.ylabel("Mean DMN-insula connectivity, Fisher z")
plt.title("DMN-insula connectivity by anxiety group")
plt.tight_layout()
plt.show()

```



1.14 11. Biological interpretation

If the real data showed a positive relationship, a cautious interpretation would be:

Higher trait anxiety is associated with stronger coupling between internally oriented DMN regions and insula regions involved in interoception and salience processing.

Possible biological meaning:

- Anxious individuals may show stronger linking between internal self-focused thought and bodily/salience signals.
- The insula may act as a bridge between bodily feeling and large-scale network switching.
- Stronger DMN–insula coupling could reflect more internally focused monitoring, rumination, or anticipation.

1.14.1 Important caution

Functional connectivity is correlational.

It does **not** prove:

- the insula causes anxiety
- the DMN causes anxiety
- stronger connectivity is necessarily bad
- the result is clinically diagnostic

A stronger paper would include:

- motion controls
- age/sex controls
- multiple comparison correction
- replication dataset
- preregistered ROI definitions
- robustness checks with different atlases

1.15 12. Save results for later

Always save the table you created.

This lets you:

- inspect results in LibreOffice/Excel
- make better figures later
- run alternative models without recomputing everything
- document exactly what was analyzed

```
[17]: # =====  
# 12. Save output  
# =====  
  
OUT_DIR = Path("outputs")  
OUT_DIR.mkdir(exist_ok=True)  
  
results_path = OUT_DIR / "dmn_insula_trait_anxiety_results.csv"  
fc_results.to_csv(results_path, index=False)  
  
print("Saved:", results_path.resolve())  
fc_results.head()
```

Saved: /home/john-walkey/Data/Research/connectome/notebooks/outputs/dmn_insula_trait_anxiety_results.csv

```
[17]: participant_id  stai_trait  anxiety_group  dmn_insula_r  dmn_insula_z
0      sub-001    46.967142      high          0.070911      0.071030
1      sub-002    40.617357      middle        0.107560      0.107978
2      sub-003    48.476885      high          0.205475      0.208443
3      sub-004    57.230299      high          0.191802      0.194207
4      sub-005    39.658466      middle        0.126784      0.127470
```

2 What you learned

This notebook transformed a research idea into a pipeline:

Idea:

Trait anxiety may involve altered interaction between internally oriented cognition and salience/interoceptive systems.

Operationalization:

Use STAI trait anxiety score and DMN-insula functional connectivity.

Pipeline:

Metadata → subjects → fMRI/BIDS → ROIs → time series → FC matrix → statistics → visualization → interpretation.

Next real-data step:

Download a real BIDS dataset with STAI trait scores, inspect the exact metadata column names, choose a justified atlas, and replace the simulated time series with real preprocessed BOLD extraction.